

C++基础：基本数据类型 与导出数据类型

目录

- Hello World
 - 安装运行环境: [vscode/docker/linux+g++/win+wsl2](#)
 - 基本元素: 注释/头文件/函数/语句/标准输入输出流
 - 基本符号与ASCII码
 - C++的词汇列表: 关键字/标志符/字面常量/运算符/分隔符/
 - C++程序的基本框架: 主函数/预处理命令/SP/OOP框架
 - 运行C++程序: 编辑-预处理-编译-链接-调试

VSCode: 下载

- 从官网（<https://code.visualstudio.com/>）下载，请注意vscode是开源软件，不要去其他地方下载付费软件。下载自己操作系统的版本。

The image shows the Visual Studio Code website on the left and the VS Code application interface on the right.

Visual Studio Code Website:

- Header: Visual Studio Code, Docs, Updates, Blog, API, Extensions, FAQ, Learn, Search Docs, Download.
- Main Text: Code editing. Redefined. Free. Built on open source. Runs everywhere.
- Download for Windows (Stable Build) button.
- Download links for macOS, Windows x64, and Linux x64.

		Stable	Insiders
macOS	Universal	↓	↓
Windows x64	User Installer	↓	↓
Linux x64	.deb .rpm	↓ ↓	↓ ↓

[Other downloads or open on web](#)

VS Code Application Interface:

- File, Edit, Selection, View, Go, Debug, Terminal, Help.
- EXTENSIONS: MARKETPLACE sidebar showing various extensions like Python, GitLens, C/C++, ESLint, Debugger for Chrome, Language Support, vscode-icons, Vetur, and C#.
- Main editor showing a JavaScript file (serviceWorker.js) with code for registering a service worker.
- TERMINAL panel at the bottom showing the output of the create-react-app command.

VSCode: 配置C++插件

VS Code interface showing the C/C++ extension configuration.

Left Panel (Extensions View):

- Search bar: C++
- Extension list:
 - C/C++** (Microsoft) - 494ms. This extension is highlighted with a red box.
 - C++ Intellisense (austin) - 4.4M, 3 stars. Install button.
 - C/C++ Extension Pack (Microsoft) - Popular extensions for C++ ...
 - Better C++ S... (Jeff Hykin) - 10.1M, 5 stars. Install button.
 - C/C++ Themes (Microsoft) - UI Themes for C/C++ extens...

Right Panel (C/C++ Extension Details):

- Extension: C/C++ - CPP-Course-Demo - Visual Studio Code
- Version: v1.14.3
- Developer: Microsoft
- Downloads: 42,941,725
- Rating: 4.5 stars (507)
- Description: C/C++ IntelliSense, debugging, and code browsing.
- Buttons: Disable, Uninstall, Settings
- Status: This extension is enabled globally.
- Navigation: DETAILS, FEATURE CONTRIBUTIONS, CHANGELOG, RUNTIME STATUS
- Section: C/C++ for Visual Studio Code
- Links: Repository, Issues, Documentation, Code Samples
- Live Share: enabled

Bottom Panel:

- PROBLEMS (1)
- OUTPUT
- TERMINAL
- DEBUG CONSOLE

安装C++编译器

- <https://sourceforge.net/projects/mingw-w64/files/mingw-w64/mingw-w64-release/>

 **SOURCEFORGE**

Open Source SoftwareBusiness SoftwareResources

Sync you to S

Home / Browse / Software Development / Compilers / MinGW-w64 - for 32 and 64 bit Windows / Files



MinGW-w64 - for 32 and 64 bit Windows Files

A complete runtime environment for gcc
Brought to you by: [jon_y](#), [ktietz70](#), [nightstrike](#)

SummaryFilesReviewsSupportMailing ListsTickets ▾NewsDiscussionDonate CodeGit ▾Wiki2

 **Download Latest Version**
mingw-w64-v10.0.0.zip (16.0 MB)

Get Updates



Home / mingw-w64 / mingw-w64-release

Name ▾	Modified ▾	Size ▾	Downloads / Week ▾
 Parent folder			
mingw-w64-v10.0.0.zip	2022-04-04	16.0 MB	47,483  
mingw-w64-v10.0.0.tar.bz2	2022-04-04	9.6 MB	6,560  

配置linux+g++

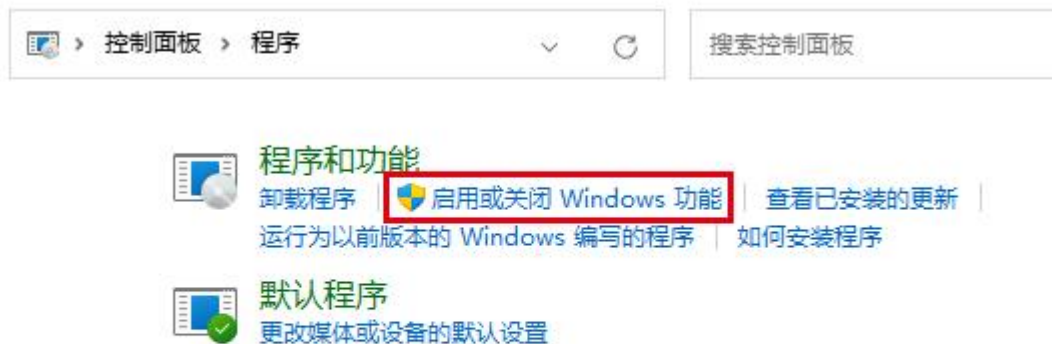
- 如果你有linux操作系统，一般自带了g++，如果没有可以用命令行安装，以ubuntu为例：
- `sudo apt update`
- `sudo apt install -y g++`

```
$ sudo apt install g++
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
g++ is already the newest version (4:11.2.0-1ubuntu1).
g++ set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 142 not upgraded.
$ |
```

- 如果是MacOS，可以直接用clang编译器。

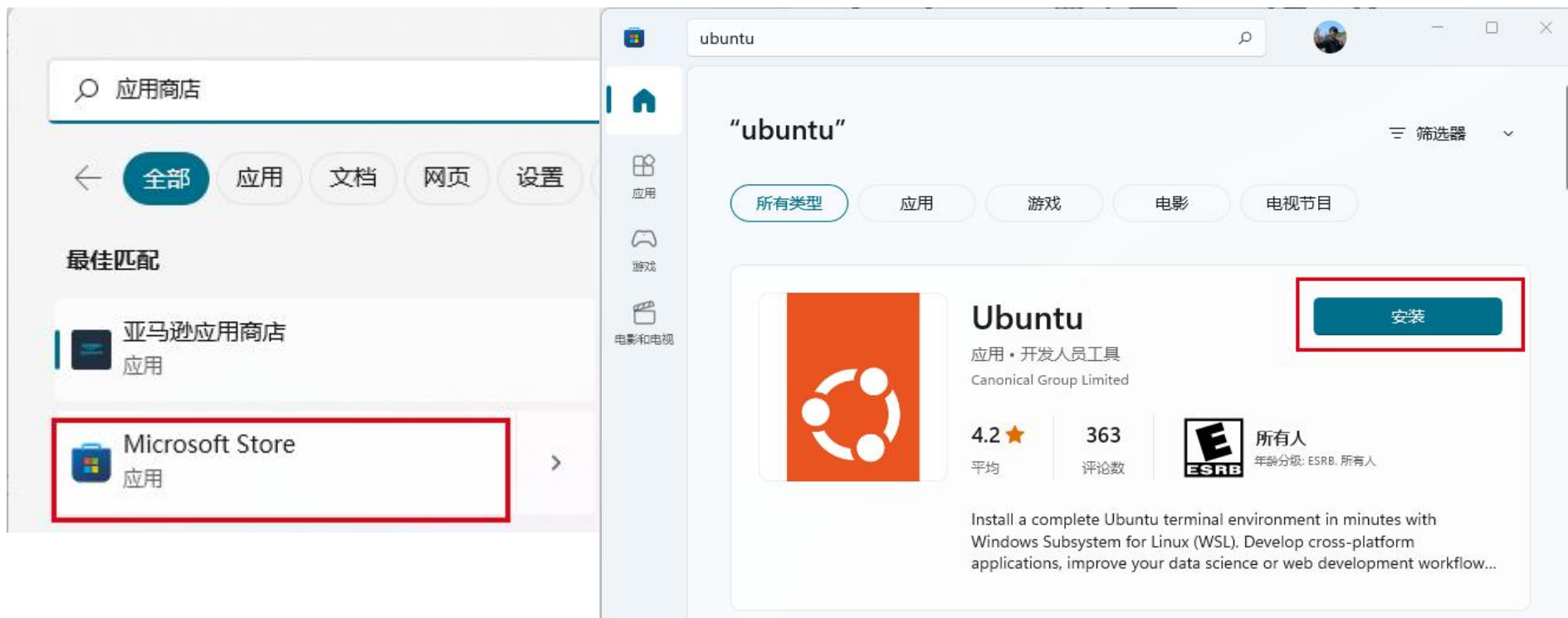
windows配置wsl2+linux+g++

- windows可以通过wsl2安装linux子系统，首先需要打开控制面板->程序和功能->启用或关闭windows功能，打开对应的选项，确定后重启就启动了wsl2。



windows配置wsl2+linux+g++

- 重启后去windows应用商店安装ubuntu系统



windows配置wsl2+linux+g++

- 安装完成后，就可以在windows上打开ubuntu系统，然后其他步骤与linux上一样，安装g++（一般已经自带了），然后编译源码就可以了。
- 本课程的案例基于windows系统进行编译运行。

目录

- Hello World
 - 安装运行环境：vscode/docker/linux+g++/win+wsl2
 - 基本元素：注释/头文件/函数/语句/标准输入输出流
 - 基本符号与ASCII码
 - C++的词汇列表：关键字/标志符/字面常量/运算符/分隔符/
 - C++程序的基本框架：主函数/预处理命令/SP/OOP框架
 - 运行C++程序：编辑-预处理-编译-链接-调试
- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

基本元素：注释

- 双斜杠是C++程序中的注释符号，写在//后面的所有内容都被视作注释；如果需要嵌入注释可以用注释内容，/*中间的所有内容都是注释内容*/。
- 注释掉的内容注释被当作空格处理，在编译阶段不会参与编译，提高代码的可读性。

```
// helloworld.cpp
/*
author: Li Zhengdan(lzd@nankai.edu.cn)
date: 2025/02
function: 打印Hello World, 无返回值
*/
#include <iostream>
using namespace std;
int main() {
    cout<<"Hello World!"<<endl;
    return 0;
}
```

```
$ g++ helloworld.cpp -o helloworld.exe
$ ./helloworld.exe
Hello World!
```

基本元素：头文件

- `#include` 是一条编译预处理指令，它告诉编译系统在编译本程序时把系统提供的头文件 `iostream.h` 的内容插入到程序中。
- 头文件就是我们编程时需要引入的别的程序文件，头文件本身也是一个C++源码程序。
- 通过`#include`<添加头文件>，我们可以方便的使用别人为我们写好的许多组件，这就是所谓的，大树底下好乘凉，或者说，站在巨人的肩膀上。

```
// helloworld.cpp
/*
author: Li Zhengdan(lzd@nankai.edu.cn)
date: 2025/02
function: 打印Hello World, 无返回值
*/
#include <iostream>
using namespace std;
int main() {
    cout<<"Hello World!"<<endl;
    return 0;
}
```

```
$ g++ helloworld.cpp -o helloworld.exe
$ ./helloworld.exe
Hello World!
```

基本元素：函数与主函数

- 程序的主体由一个函数组成。其中 `main` 是函数名，`int` 指出该函数的返回类型，括号内是函数的参数，`void` 表示参数为空，等价于啥也不写。
- `main` 函数比较特殊，称为主函数，是所有程序的启动入口，也就是说，启动程序后第一个执行的是 `main` 函数。

```
// helloworld.cpp
/*
author: Li Zhengdan(lzd@nankai.edu.cn)
date: 2025/02
function: 打印Hello World, 无返回值
*/
#include <iostream>
using namespace std;
int main() {
    cout<<"Hello World!"<<endl;
    return 0;
}
```

```
$ g++ helloworld.cpp -o helloworld.exe
$ ./helloworld.exe
Hello World!
```

基本元素：语句

- 函数体用 { } 括起来，函数体内可以包含任意多行语句。
- 用{}起来的所有语句称为一个语句块。值得注意的是每一个语句块都构成一个独立的运行域，即外部语句无法访问位于语句块内定义的变量。

```
// helloworld.cpp
/*
author: Li Zhengdan(lzd@nankai.edu.cn)
date: 2025/02
function: 打印Hello World, 无返回值
*/
#include <iostream>
using namespace std;
int main() {
    cout<<"Hello World!"<<endl;
    return 0;
}
```

```
$ g++ helloworld.cpp -o helloworld.exe
$ ./helloworld.exe
Hello World!
```


基本元素：标准输入输出流

- `cout` 是一个标准输出文件名，这里表示屏幕。符号“<<”是运算符，它指示计算机把其右端用双引号括起来的字符串输送到`cout`文件即屏幕（或打印机）。
- `cout`和“<<”符号的说明都在系统提供的头文件 `iostream.h`之中，因此，凡是程序中需要使用`cout`和“<<”等标准输入输出机制时，第 2 行包含的指令就必须列出。

```
// helloworld.cpp
/*
author: Li Zhengdan(lzd@nankai.edu.cn)
date: 2025/02
function: 打印Hello World, 无返回值
*/
#include <iostream>
using namespace std;
int main() {
    cout<<"Hello World!"<<endl;
    return 0;
}
```

```
$ g++ helloworld.cpp -o helloworld.exe
$ ./helloworld.exe
Hello World!
```

基本元素：标准输入输出流

- `cin` 与 `cout` 类似，是标准输入文件（指键盘）的名字。“>>”称作提取运算符。以下的 C++ 程序允许从键盘输入一个整数，例如 29，结果在屏幕上显示。
- `myage` 是一个程序员定义的整型变量名，变量不同于常量，在程序运行中其值可以改变。该语句的作用是把从键盘输入的一个整数赋给变量 `myage`。

course1 >  cin_test.cpp > ...

```
1  //program2-3.cpp
2  #include <iostream>
3  using namespace std;
4  int main(void)
5  {
6      int myage;
7      cout<<"My age is ";
8      cin>>myage; //输入年龄(一个整数)
9      cout<<endl;
10     return 0;
11 }
```

```
$ g++ cin_test.cpp -o cin_test.exe
$ ./cin_test.exe
My age is 29
```

复杂的程序实例

- 是求解一道中国古代数学题。已知公鸡 5 元一只，母鸡 3 元一只，雏鸡 3 只 1 元，问花 100 元买 100 只鸡，应各有几只。这个问题可能有多个答案。

```
course1 >  count_chicken.cpp > ...
```

```
1  //count_chicken.cpp
2  #include <iostream>
3  using namespace std;
4  int main(void)
5  {
6      int chicken, hen, cock;
7      cout<<"chicken hen cock"<<endl;
8      for(chicken=0; chicken<100; chicken+=3)
9          for(hen=0; hen<=33; hen++)
10             if((cock=100-chicken-hen)> -1)
11                 if(chicken/3+hen* 3+cock* 5==100)
12                     cout<<" "<<chicken<<" "<<hen<<" "<<cock<<endl;
13     return 0;
14 }
```

```
$ g++ count_chicken.cpp -o count_chicken.exe
$ ./count_chicken.exe
chicken hen cock
75 25 0
78 18 4
81 11 8
84 4 12
```

基本元素：复杂的程序实例

- (1) 由#include开头的2行是编译预处理命令，用于输入输出数据。
- (2) 由int main(void)开头的主函数，它是整个程序的主体。
- (3) 程序定义了3个变量：chicken表示雏鸡的个数，hen表示母鸡的个数，cock表示公鸡的个数。变量的概念将在后面介绍。

```
2  #include <iostream>
3  using namespace std;
4  int main(void)
5  {
6      int chicken, hen, cock;
7      cout<<"chicken hen cock"<<endl;
8      for(chicken=0; chicken<100; chicken+=3)
9          for(hen=0; hen<=33; hen++)
10             if((cock=100-chicken-hen)> -1)
11                 if(chicken/3+hen* 3+cock* 5==100)
12                     cout<<" "<<chicken<<" "<<hen<<" "<<cock<<endl;
13     return 0;
14 }
```

```
$ g++ count_chicken.cpp -o count_chicken.exe
$ ./count_chicken.exe
chicken hen cock
75 25 0
78 18 4
81 11 8
84 4 12
```

基本元素：复杂的程序实例

- (4) 第 7、12 行是类似上例的输出语句，不过这里更为复杂，其中第 11 行的输出语句被执行了多次。
- (5) 第 8~11 行包含 for（循环）语句和 if（分支）语句，用来对不同数量的雏鸡、公鸡、母鸡分别进行判断，凡是满足“百鸡百钱”条件的数值都作为结果输出。具体算法是，令 chicken 取值 0~100，hen 取值 0~33，令 cock 取值为 $\text{cock} = 100 - \text{chicken} - \text{hen}$ ，在这样一个范围内搜索符合条件的解，这个条件就是： $\text{chicken}/3 + \text{hen} * 3 + \text{cock} * 5 = 100$ 。
- 有关 for（循环）语句和 if（分支）语句的用法在第四章介绍。

```
6      int chicken, hen, cock;
7      cout << "chicken hen cock" << endl;
8      for(chicken=0; chicken<100; chicken+=3)
9          for(hen=0; hen<=33; hen++)
10             if((cock=100-chicken-hen)> -1)
11                 if(chicken/3+hen* 3+cock* 5==100)
12                     cout << " " << chicken << " " << hen << " " << cock << endl;
```

基本元素： 复杂的程序实例

- (6) 这个 C++ 程序的执行结果将在屏幕上输出：

```
$ g++ count_chicken.cpp -o count_chicken.exe
$ ./count_chicken.exe
chicken hen cock
75 25 0
78 18 4
81 11 8
84 4 12
```

结果说明这个题有 4 组解，其正确性不难验证。

目录

- Hello World
 - 安装运行环境：vscode/docker/linux+g++/win+wsl2
 - 基本元素：注释/头文件/函数/语句/标准输入输出流
 - 基本符号与ASCII码
 - C++的词汇列表：关键字/标志符/字面常量/运算符/分隔符/
 - C++程序的基本框架：主函数/预处理命令/SP/OOP框架
 - 运行C++程序：编辑-预处理-编译-链接-调试
- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

基本符号与ASCII码

- 所有的 C++ 程序都是一个按 C++ 语法规则组织的由基本符号组成的序列基本符号分类如下：

1. 字母

由大小写英文字母

A, B, C, ... , X, Y, Z

a, b, c, ... , x, y, z

共 52 个符号组成第 1 类基本符号。

2. 数字

0, 1, 2, ... , 9

共 10 个符号组成第 2 类基本符号。

3. 特殊符号

␣、!、#、\$、%、&、,、'、"、(、)、*、+、'、-、•、/、:、;、

<、=、>、?、@、[、\、]、^、_、{、|、}、~

共 33 个符号组成第 3 类基本符号（␣表示空格）。

基本符号与ASCII码

- 计算机本身不能直接区分不同的字母、数字或特殊符号，它是根据每个符号对应的编码来识别这些基本符号的。这些符号的编码表称为ASCII码表。
- ASCII 是美国标准信息交换码（American Standard Code for Information Interchange）的英文缩写，ASCII 码表把 95 个基本（可打印）符号和 33 个控制字符共 128 个字符与 7 位二进制数 0000000~1111111 共 128 个数码建立了对应关系，实际上任何一个基本符号在计算机内的表示形式就是这样一个二进制数码。

表 2.1 ASCII 码表

代码	字符	名称	代码	字符	名称	代码	字符	名称	代码	字符	名称
000	NUL	无效	032		空格	064	@		096	'	单引
001	SOH	标题始	033	!	叹号	065	A		097	a	
002	STX	正文始	034	"	双引	066	B		098	b	
003	ETX	正文尾	035	#	#号	067	C		099	c	
004	EOT	传递止	036	\$	币号	068	D		100	d	
005	ENQ	查询	037	%	百分号	069	E		101	e	
006	ACK	信号确认	038	&	与号	070	F		102	f	

代码	字符	名称	代码	字符	名称	代码	字符	名称	代码	字符	名称
007	BEL	响铃	039	'	单引	071	G		103	g	
008	BS	退格	040	(	圆括号	072	H		104	h	
009	HT	水平制表	041	)	圆括号	073	I		105	i	
010	LF	换行	042	*	乘号	074	J		106	j	
011	VT	垂直制表	043	+	加号	075	K		107	k	
012	FF	换页	044	,	逗号	076	L		108	l	
013	CR	回车	045	-	减号	077	M		109	m	
014	SO	移出	046	•	圆点	078	N		110	n	
015	SI	移入	047	/	除号	079	O		111	o	
016	DLE	换码	048	0		080	P		112	p	
017	DC1	设备控制	049	1		081	Q		113	q	
018	DC2	设备控制	050	2		082	R		114	r	
019	DC3	设备控制	051	3		083	S		115	s	
020	DC4	设备停机	052	4		084	T		116	t	
021	NAK	信息否认	053	5		085	U		117	u	
022	SYN	同步	054	6		086	V		118	v	
023	ETB	块传送止	055	7		087	W		119	w	
024	CAN	作废	056	8		088	X		120	x	
025	EM	纸用完	057	9		089	Y		121	y	
026	SUB	置换	058	:	冒号	090	Z		122	z	
027	ESC	换码	059	;	分号	091	[	方括号	123	{	括号
028	FS	文件分离	060	<	小于	092	\	反斜杠	124		竖线
029	GS	分组	061	=	等于	093	]	方括号	125	}	括号
030	RS	记录分离	062	>	大于	094	^		126	~	波纹
031	US	元素分离	063	?	问号	095	_	下横线	127	DEL	删除

基本符号与ASCII码

- 从 032~126 共 95 个可打印字符构成了基本字符集。
- 计算机的基本存储单元是字节，一个字节有 **8 个二进制位**，一般一个基本字符的代码存为一个字节。用 8 位编码可表示 256 个字符，因此在不同的计算机中有不同的扩展 ASCII 码表，在扩展的 ASCII 码表中，128~255 号字符一般是不同语言的特殊字符、特殊制表符、及其他特殊符号。在许多高级语言（如 Basic、C）的书中，可以找到扩展的 ASCII 码表，其中扩展部分只供读者参考，因为其符号与编码的对应可能不是唯一的。
- 比如我们常用文字表情，就是一种特殊的字符编码。

表情符号与人物



ABC



空格



目录

- Hello World
 - 安装运行环境：vscode/docker/linux+g++/win+wsl2
 - 基本元素：注释/头文件/函数/语句/标准输入输出流
 - 基本符号与ASCII码
 - C++的词汇列表：关键字/标志符/字面常量/运算符/分隔符/
 - C++程序的基本框架：主函数/预处理命令/SP/OOP框架
 - 运行C++程序：编辑-预处理-编译-链接-调试
- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

C++词汇列表：关键字

- 关键字（key word）是一类有特定的专门含义的单词。对于 C++ 语言来说，**凡是列入关键字表的单词，一律不得移作它用**。因此，关键字又称为保留字（reserved word）。在上节的程序中，void、int、for、if 等单词就属于关键字。
- 在使用 C++ 语言编程前应首先弄清楚关键字集合，以免在编程中产生错误，至少应**避免在设置标识符时与关键字重名**。
- 在IDE中进行编码编辑的时候，关键字一般会用**特殊颜色**进行标记，此外，如果使用关键字作为变量，编译器也会提示出错。

course1 >  helloworld.cpp > ...

```
1  // helloworld.cpp
2  #include <iostream>
3  using namespace std;
4  int main(void)
5  {
6      cout<<"Hello World!"<<endl;
7      return 0;
8  }
```

C++词汇列表：关键字

- 基本的关键字表如下所示，这些关键词将陆续出现在以后的章节中（**掌握**）。

表 2.2 基本关键字表

asm	auto	bool	break
case	catch	char	class
const	continue	default	delete
do	double	else	enum
extern	false	float	for
friend	goto	if	inline
int	long	namespace	new
operator	private	protected	public
register	return	short	signed
sizeof	static	struct	switch
template	this	throw	true
try	typedef	union	unsigned
using	virtual	void	volatile
wchar_t	while		

C++词汇列表：标识符

- 标识符（**identifier**）是由程序员为程序中的各种成分，如：变量、有名常量、用户定义的类型、枚举类型的值、函数及其参数、类、对象等所起的名字。名字不能随便起，必须符合标识符的组成规则：
 - 标识符是一个以字母或下横线“**_**”开头的，由字母、数字、下横线组成的字符串，如：**abcd**、**c5**、**PERSON_H**等都是合法的标识符，而**3A**、**A* B**、**\$ 43.5A**等都是不合法的，一个标识符中间不可插入空格。
 - 标识符应与任一关键字有区别，如**for**、**if**、**case**等都不可用做标识符。
 - 标识符中字母区分大小写，如**Abc**与**abc**被认为是不同的两个标识符。关键字字母的大小写，不同系统可能有不同的规定，如**C++**中的关键字用小写字母，而**FOR**、**For**、**foR**可以作为标识符使用。
 - 标识符的有效长度也有规定。如果程序中的标识符过长，系统将对有效长度之外的字符忽略不计，一般**C++**语言设其有效长度为**32**。
- 有许多关于标识符定义的问题和考题，这几条规则一定要遵守。

C++词汇列表：标识符

- 有许多关于标识符定义的问题和考题，上述规则一定要遵守。
- 用户在为常量、变量、函数等起名字时，往往不是简单地用 a、b、c、n1、n2...这样的名字，而**要使标识符有一定的描述性**，比如表示母鸡数量的变量名为 **hen**，表示我的年纪的变量名为 **myage** 或 **my_age**、**myAge**。有许多关于标识符命名的规则，各位同学可以去网上找更多资料，比如驼峰命名法。

course1 >  cin_test.cpp > ...

```
1  //program2-3.cpp
2  #include <iostream>
3  using namespace std;
4  int main(void)
5  {
6      int myage;
7      cout<<"My age is ";
8      cin>>myage; //输入年龄(一个整数)
9      cout<<endl;
10     return 0;
11 }
```

C++词汇列表：字面常量

- C++程序中的常量是指固定不变的量，有两种表示形式：一种称为有名常量，一种称为字面常量（literal constant）。例如圆周率 `float pai=3.1416`，其中 `pai` 就是一个有名常量，`pai` 是量 3.1416 的名字，而 3.1416 称为字面常量。
- 字面常量分为 4 类：`int` 型常量、`float` 型常量、`char` 型常量和字符串常量。字面常量是我们可以输入程序的量，简单来说，如果我们想给程序输入内容，只能是这 4 种类型的量。

course1 >  helloworld.cpp > ...

```
1  // helloworld.cpp
2  #include <iostream>
3  using namespace std;
4  int main(void)
5  {
6      cout<<"Hello World!"<<endl;
7      return 0;
8  }

8  for(chicken=0; chicken<100; chicken+=3)
9      for(hen=0; hen<=33; hen++)
10         if((cock=100-chicken-hen)> -1)
```

course1 >  cin_test.cpp > ...

```
1  //program2-3.cpp
2  #include <iostream>
3  using namespace std;
4  int main(void)
5  {
6      int myage;
7      cout<<"My age is ";
8      cin>>myage; //输入年龄(一个整数)
```

C++词汇列表：字面常量

- `int` 型常量即整型常量，实际上就是整数。
- `float` 型常量即浮点常量，浮点常量有两种表示法：
 - 小数点表示法：4.75、2.0、-473.385。
 - 科学表示法：1.2e35、-7.37e-3

course1 >  literal_constant.cpp > ...

```
1  // literal_constant.cpp
2  #include <iostream>
3  using namespace std;
4  int main(void)
5  {
6      int radiu = 4;
7      float pai = 3.1415926535;
8      float weight = 6.9e4;
9      cout<<" radiu = "<<radiu<<"\n pai = "<<pai<<"\n weight = "<<weight<<"g\n";
10     return 0;
11 }
```

```
$ g++ literal_constant.cpp -o literal_constant.exe
$ ./literal_constant.exe
radiu = 4
pai = 3.14159
weight = 69000g
```

C++词汇列表：字面常量

- char 常量即字符常量，如：'A'、'g'、'3'、'!'。C++语言还定义了一些特殊的字符常量，全是用反斜杠“\”开头的，如下(要记住)

表 2.3 字符常量的表示及含义

字 符 常 量	ASCII 码	含 义	符 号
\a	007	响铃	BEL
\b	008	退格	BS
\f	012	走纸	FF
\n	010	换行	LF
\r	013	回车	CR
\t	111	水平制表	HT
\v	011	垂直制表	VT
\\	092	反斜杠	\
\'	044	单引号	'
\"	034	双引号	"
\?	063	问号	?
\0	048	整数 0	NUL
\DDD	0DDD	八进制整数	
\xHHH	0xHHH	十六进制整数	

C++词汇列表： 字面常量

- 字符串常量是用双引号括起来的字符序列，称为字符串常量。比如"string constant"。

course1 >  helloworld.cpp > ...

```
1 // helloworld.cpp
2 #include <iostream>
3 using namespace std;
4 int main(void)
5 {
6     cout<<"Hello World!"<<endl;
7     return 0;
8 }
```

course1 >  literal_constant.cpp >  main(void)

```
1 // literal_constant.cpp
2 #include <iostream>
3 using namespace std;
4 int main(void)
5 {
6     int radiu = 4;
7     float pai = 3.1415926535;
8     float weight = 6.9e4;
9     const char* str = "Hello World! \nHello China! \0 Hello Nankai\n";
```

course1 >  cin_test.cpp > ...

```
1 //program2-3.cpp
2 #include <iostream>
3 using namespace std;
4 int main(void)
5 {
6     int myage;
7     cout<<"My age is ";
8     cin>>myage; //输入年龄(一个整数)
9     cout<<endl;
10    return 0;
11 }
```


C++词汇列表：字面常量

- 一个字符串常量是一个特殊的字符序列或字符数组，其长度为该字符串中所有字符的个数加 1。原因是除了保存串中字符（包括空格）之外，**在最后存一串尾符'\0'**。例如，字符串常量"Pascal"的长度为 7。
- 字符串中也可包括特殊字符，如下。不过在使用特殊符号'\0' 时，会产生问题，系统将把它作为串尾符而忽略了它后面的其他字符。

course1 >  literal_constant.cpp >  main(void)

```
1 // literal_constant.cpp
2 #include <iostream>
3 using namespace std;
4 int main(void)
5 {
6     int radiu = 4;
7     float pai = 3.1415926535;
8     float weight = 6.9e4;
9     const char* str = "Hello World! \nHello China! \0 Hello Nankai\n";
10    cout<<" radiu = "<<radiu<<"\n pai = "<<pai<<"\n weight = "<<weight<<"g\n";
11    cout<< str<<endl;
12    return 0;
13 }
```

```
$ g++ literal_constant.cpp -o literal_constant.exe
$ ./literal_constant.exe
radiu = 4
pai = 3.14159
weight = 69000g
Hello World!
Hello China!
```

C++词汇列表：运算符

- 运算符（operator）主要由字母、数字之外的第 3 类基本符号组成，少数的例外是个别关键字，如 **sizeof**、**new**、**delete**，也被列入运算符之列，其余运算符为：

+、-、*、/、%、=、!=、<、<=、>、>=、!、&&、||、&、^、|、~、++、--、+=、-=、*=、/=、%=、<<=、>>=、&=、^=、|=、? :、=、()、[]、*、->、<<、>>、'、::

- 运算符的概念和用法来自数学，不过在 C++ 语言中运算符和运算的概念对于人们日常习惯的理解已有所扩展。例如，在 C++ 语言中，赋值（“=”），表达式的并列（<表达式>，<表达式>），a[i] 中表示下标的[]都一律作为运算和运算符来处理。
- 注意：符号运算的优先级将在后面讲解

C++词汇列表：分隔符

- 分割符本身没有明确的含义，但程序中却必不可少，一般用来界定或分割其他语法成分的单词称为分割符（separator）。程序中的分割符有点像文章中的标点符号。例如：
 - “;”：表示一个语句的结束。
 - “”：表示一个字符串的开始与结束
- 分割符包括：
 - `␣`（空格）、`"`、`#`、`(`、`)`、`/`、`*`、`*`、`/`、`//`、`'`、`;`、`{`、`}`

course1 >  helloworld.cpp > ...

```
1  // helloworld.cpp
2  #include <iostream>
3  using namespace std;
4  int main(void)
5  {
6      cout<<"Hello World!"<<endl;
7      return 0;
8  }
```

C++词汇列表：分隔符

- 较为特殊的是**空格**，程序中空格的`使用`十分重要。`C++`程序允许连续的空格出现，从语法功能看，连续多个空格与一个空格作用相同，在两个相邻的关键字或标识符之间起到了分割的作用。另外，在程序中连续的空格可改善程序的格式，提高程序的可读性。
- 分割符“`/*`”、“`* /`”和“`//`”用于注释。“`/*`”与“`* /`”应成对地出现，其间的任何符号序列在程序中等价于一个空格，这为程序员在程序中加入注释以提高可读性提供了方便。分割符“`//`”有类似的功能，它使其后直至行尾的任何字符序列成为注释。
- 分割符“`,`”`，`”可用来分割并列的变量、对象、参数等，**同时“`,`”`，`也是运算符。**

course1 >  sign_test.cpp > ...

```
1  // sign_test.cpp
2  #include <iostream>
3  using namespace std;
4  int main(void)
5  {
6      int var = (1, 2, 3);
7      cout<<"var = "<<var<<endl;
8      return 0;
9  }
```

```
$ g++ sign_test.cpp -o sign_test.exe
$ ./sign_test.exe
var = 3
```

目录

- Hello World
 - 安装运行环境：vscode/docker/linux+g++/win+wsl2
 - 基本元素：注释/头文件/函数/语句/标准输入输出流
 - 基本符号与ASCII码
 - C++的词汇列表：关键字/标志符/字面常量/运算符/分隔符/
 - **C++程序的基本框架：主函数/预处理命令/SP/OOP框架**
 - 运行C++程序：编辑-预处理-编译-链接-调试
- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

C++程序的基本框架： 主函数

- 以 **main** 命名的主函数是 C++ 程序中具有特殊性质和功能的函数，**main()** 函数是整个程序的入口。
- 主函数是任何一个 C++ 程序中唯一必不可少的函数。一个最短的合法 C++ 程序为： **int main(){ }**。
- 主函数的函数名是标识符 **main**，它是由系统指定的。注意 **main** 不是关键字。

```
course1 >  main_test.cpp > ...
```

```
1 //main_test.cpp|
2 int main(){}
```

```
$ g++ main_test.cpp -o main_test.exe
$ ./main_test.exe
```


C++程序的基本框架：主函数

- 主函数的类型（返回类型）为int 型（一些编译器可以为void），它可以返回一个整型数值，这个值是传送给操作系统的。所以当你在python中运行C++程序并获取返回值时，可以用main函数返回。
- 主函数可以带参数：int main(int argc, char * argv[])，主函数的形参类型也是由系统规定的，用户不得随意设计。但是形参名是可以改变的，比如我的例子中就用了argcount和argvalues。

```
course1 >  main_test.cpp >  main(int, char * [])
```

```
1 //main_test.cpp
2 #include<iostream>
3 using namespace std;
4 int main(int argcount, char* argvalues[]){
5     for(int i = 0; i<argcount; i++){
6         cout<<"arg["<<i<<"]: "<<argvalues[i]<<endl;
7     }
8 }
```

```
$ g++ main_test.cpp -o main_test.exe
$ ./main_test.exe Hello Nankai University
arg[0]: ./main_test.exe
arg[1]: Hello
arg[2]: Nankai
arg[3]: University
```

C++程序的基本框架： 主函数

- 当主函数形式为 `int main(int argc, char* argv[])` 时：
- **argc**: 存放命令行中字符串的个数，即命令名加上参数的个数。它总是大于或等于1（等于1的时候相当于没有输入参数）。
- **argv**: 字符串数组指针。保存了命令行中输入的各个字符串。`argv[0]`指向命令行的第一个字符串（命令名），`argv[1]`指向命令行第2个字符串（第1个参数），以此类推。

```
course1 >  main_test.cpp >  main(int, char * [])
```

```
1  //main_test.cpp
2  #include<iostream>
3  using namespace std;
4  int main(int argc, char* argv){
5      for(int i = 0; i<argc; i++){
6          cout<<"arg["<<i<<"]: "<<argv[i]<<endl;
7      }
8  }
```

```
$ g++ main_test.cpp -o main_test.exe
$ ./main_test.exe Hello Nankai University
arg[0]: ./main_test.exe
arg[1]: Hello
arg[2]: Nankai
arg[3]: University
```

C++程序的基本框架： 主函数

- 主函数可调用任何其他函数，但它本身不可由任何函数调用，而只能由 C++ 程序被编译后的执行代码将要在其上运行的操作系统自动调用，实际上它是为系统运行该程序标识出启动地址。因此，主函数也是任一 C++ 程序运行的执行入口。
- 主函数不可作其他属性说明，如不可说明为静态（**static**）的、内联（**inline**）的，等等。

```
course1 >  main_test.cpp >  main(int, char * [])
```

```
1 //main_test.cpp
2 #include<iostream>
3 using namespace std;
4 int main(int argc, char* argv[]){
5     for(int i = 0; i<argc; i++){
6         cout<<"arg["<<i<<"]: "<<argv[i]<<endl;
7     }
8 }
```

```
$ g++ main_test.cpp -o main_test.exe
$ ./main_test.exe Hello Nankai University
arg[0]: ./main_test.exe
arg[1]: Hello
arg[2]: Nankai
arg[3]: University
```

C++程序的基本框架：预处理命令

- C++语言的编译系统包含一个附加的编译预处理程序，它是一个文本处理程序，在源程序被正式编译之前，首先进行预处理。其功能是读取和识别程序中的预处理命令，并按预处理命令对编译的程序进行指定的预处理。
- 编译预处理命令都以符号“#”开头，指出在对该程序进行编译之前应做的“预处理”工作。
- 预处理主要包括以下几个命令：
 - 文件嵌入命令（#include）
 - 宏定义与取消宏定义命令（#define/#undef）：
 - 条件编译命令：
 - #if 和#endif
 - #ifdef 和#endif
 - #ifndef 和#endif
 - #elif 和#else， #endif

C++程序的基本框架：预处理命令

- 文件嵌入命令（`#include`）：要求系统在编译之前把它指明的文件嵌入到该命令所在位置。其格式为：
 - `#include <<文件名>>`
 - `#include "<文件名>"`
- 当头文件用<>括起来时，要求系统首先在由系统提供的 **include** 目录下搜索该文件。当头文件用双引号括起来时，要求系统首先在源程序所在的当前目录中查找。在当前目录未找到该文件时，再转到包含的目录中查找。

```
course1 >  preprocess_test.cpp > ...  
1 //preprocess_test.cpp  
2 #include "helloworld.h"  
3 #include <iostream>  
4 using namespace std;  
5 int main(){  
6     helloworld();  
7     return 0;  
8 }
```

```
course1 >  helloworld.h > ...
```

```
1 //helloworld.h  
2 #include<iostream>  
3 using namespace std;  
4  
5 void helloworld(){  
6     |    cout<<"Hello World!"<<endl;  
7     |  
8 }
```

```
$ g++ preprocess_test.cpp -o preprocess_test.exe  
$ ./preprocess_test.exe  
Hello World!
```


C++程序的基本框架：预处理命令

- 每一 **include** 命令只指定一个包含文件。
- 文件的包含可以嵌套，即被包含文件中又有 **include** 命令。这时一个嵌入命令实际上可包含多个文件。**应避免互相包含的情形，可能会出现循环嵌套和变量函数重复定义的问题。**
- 文件的包含把两个或多个程序模块组合成一个源程序，作为一个整体进行编译，它与连接（link）的概念不同。
- **include** 命令不一定只把扩展名为.h 的头文件包含进来，也可以是.cpp 或其他文件。头文件应该以说明语句为主，**谨慎定义变量。**

course1 > C helloworld.h > ...

```
1 //helloworld.h
2 #include<iostream>
3 using namespace std;
4
5 void helloworld(){
6     cout<<"Hello World!"<<endl;
7 }
```

course1 >  preprocess_test.cpp > ...

```
1 //preprocess_test.cpp
2 #include "helloworld.h"
3 #include <iostream>
4 using namespace std;
5 int main(){
6     helloworld();
7     return 0;
8 }
```


C++程序的基本框架：预处理命令

- 宏定义与取消宏定义命令（`#define/#undef`）：是用一个称为宏名的标识符代表一段字符串，后者称为宏替换体。在预处理过程中，把程序中出现的所有该宏名，用替换体取而代之。其格式为：
 - `#define <宏名> <宏替换体>`
- 宏名：为一标识符，习惯上用大写字母组成，它代替宏替换体出现在程序中。
- 宏替换体：为一任意字符串。它可能是一个数字常量，可计算值的算术表达式，字符串常量也可以是一个语句
- 宏定义可以作为常量说明。例如，符号常量 YES 为整数 1，PI 为浮点常量 3.14159，其作用类似于后面介绍的常量说明，但二者是有区别的。

```
course1 >  define_test.cpp > ...
```

```
1  //define_test.h
2  #include<iostream>
3  using namespace std;
4  #define YES 1
5  #define PI 3.14159
```

C++程序的基本框架：预处理命令

- 宏名也可以出现在另一个宏定义的宏替换体中。在预处理过程中要经过两重替换。

course1 >  define_test.cpp > ...

```
1 //define_test.h
2 #include<iostream>
3 using namespace std;
4 #define YES 1
5 #define PI 3.14159
6 #define RAD PI/180
7 #define BEGIN {
8 #define END }
9 #define MAIN main
10 #define PRT1 cout<<"\n The Begin "
11 #define Max(a,b) (a>b?a:b)
12 #define REGISTER
13 #define ZERO 0.000001
14 int MAIN(void)
15 BEGIN
16     PRT1<<"max value = "<<Max(12,3)<<endl;
17     return 0;
18 END
```

```
$ g++ define_test.cpp -o define_test.exe
$ ./define_test.exe
```

```
The Begin max value = 12
```

C++程序的基本框架：预处理命令

- 宏替换体可以是符号或字符串，比如我把{}和main函数名替换了。

course1 >  define_test.cpp > ...

```
1  //define_test.h
2  #include<iostream>
3  using namespace std;
4  #define YES 1
5  #define PI 3.14159
6  #define RAD PI/180
7  #define BEGIN {
8  #define END }
9  #define MAIN main
10 #define PRT1 cout<<"\n The Begin "
11 #define Max(a,b) (a>b?a:b)
12 #define REGISTER
13 #define ZERO 0.000001
14 int MAIN(void)
15 BEGIN
16     PRT1<<"max value = "<<Max(12,3)<<endl;
17     return 0;
18 END
```

```
$ g++ define_test.cpp -o define_test.exe
$ ./define_test.exe
```

```
The Begin max value = 12
```

C++程序的基本框架：预处理命令

- 宏替换可以是可执行的表达式语句，如例中 PRT1 的定义
- 宏替换体也可以为空，它表示该宏名已经被定义过。

course1 >  define_test.cpp > ...

```
1  //define_test.h
2  #include<iostream>
3  using namespace std;
4  #define YES 1
5  #define PI 3.14159
6  #define RAD PI/180
7  #define BEGIN {
8  #define END }
9  #define MAIN main
10 #define PRT1 cout<<"\n The Begin "
11 #define Max(a,b) (a>b?a:b)
12 #define REGISTER
13 #define ZERO 0.000001
14 int MAIN(void)
15 BEGIN
16     PRT1<<"max value = "<<Max(12,3)<<endl;
17     return 0;
18 END
```

```
$ g++ define_test.cpp -o define_test.exe
$ ./define_test.exe
```

```
The Begin max value = 12
```

C++程序的基本框架：预处理命令

- 一种复杂的宏替换是带参数的宏替换，其格式为：
 - #define <宏名><形参><宏替换体>

course1 >  define_test.cpp > ...

```
1  //define_test.h
2  #include<iostream>
3  using namespace std;
4  #define YES 1
5  #define PI 3.14159
6  #define RAD PI/180
7  #define BEGIN {
8  #define END }
9  #define MAIN main
10 #define PRT1 cout<<"\n The Begin "
11 #define Max(a,b) (a>b?a:b)
12 #define REGISTER
13 #define ZERO 0.000001
14 int MAIN(void)
15 BEGIN
16     PRT1<<"max value = "<<Max(12,3)<<endl;
17     return 0;
18 END
```

```
$ g++ define_test.cpp -o define_test.exe
$ ./define_test.exe
```

```
The Begin max value = 12
```


C++程序的基本框架：预处理命令

- 一定要注意的，**宏替换是完全的、直接的字符替换，其发生在编译之前。**一定要用括号括起来，不然可能会出意料之外的错误。

```
16  #define MUL(a,b) a*b
17  int MAIN(void)
18  BEGIN
19      PRT1<<"max value = "<<Max(12,3);
20      PRT1<<"ZERO = "<<ZERO;
21      PRT1<<"MUL(2,3+4) = "<<MUL(2,3+4);
22      cout<<endl;
23      return 0;
24  END
```

```
$ g++ define_test.cpp -o define_test.exe
$ ./define_test.exe
```

```
The Begin max value = 12
The Begin ZERO = 0.001
The Begin MUL(2,3+4) = 10
```


C++程序的基本框架： 预处理命令

- 取消宏定义命令，是用来改变原有定义内容的手段。

```
13  #define ZERO 0.000001
14  #undef ZERO
15  #define ZERO 0.001
16  #define MUL(a,b) a*b
17  int MAIN(void)
18  BEGIN
19      PRT1<<"max value = "<<Max(12,3);
20      PRT1<<"ZERO = "<<ZERO;
21      PRT1<<"MUL(2,3+4) = "<<MUL(2,3+4);
22      cout<<endl;
23      return 0;
24  $ g++ define_test.cpp -o define_test.exe
   $ ./define_test.exe
```

```
The Begin max value = 12
The Begin ZERO = 0.001
The Begin MUL(2,3+4) = 10
```

C++程序的基本框架：预处理命令

- 条件编译命令：根据条件来决定哪部分程序参加编译，哪部分程序不参加编译，主要包括：
 - `#if` 和 `#endif`
 - `#ifdef` 和 `#endif`
 - `#ifndef` 和 `#endif`
 - `#elif` 和 `#else`, `#endif`

```
#if <常量表达式>
    <程序段>
#endif
```

```
#ifdef <标识符>
    <程序段>
#endif
```

```
#elif <常量表达式>
    <程序段 1>
#else
    <程序段 2>
#endif
```

course1 >  precompile_test.cpp > ...

```
1  #include <iostream>
2  #define WHITE 0
3  #define BLACK 1
4  #define COLOR WHITE
5  using namespace std;
6  int main( ){
7      #if COLOR==WHITE
8          cout<<"Select white stone "<<endl;
9      #elif COLOR==BLACK
10         cout<<"Select black stone "<<endl;
11     #else
12         #error you select color incorrectly;
13     #endif
14     return 0;
15 }
```

C++程序的基本框架： SP/OOP框架

- 结构程序设计（SP）的基本思想是**自顶向下逐步求精**，在编程时总是把一个大的复杂的任务逐步划分为若干子任务，形成用一个个相对简单的独立的程序功能模块，并用一个一个的函数来实现。
- 结构程序设计**以函数为核心**划分成若干程序模块，在函数与函数之间，通过**全局量、函数参数和函数返回值**进行数据通信。程序员在具体编写每个模块时，所面对的问题是很简单的。

C++程序的基本框架：SP/OOP框架

- 从结构程序设计的观点来讲，一个 C++ 程序由下面几个部分组成。
- **主函数：**它是整个程序的主控模块，也是整个程序的入口和出口。
- **用户定义函数：**它由一个或若干个函数定义组成，大的程序可以有几十甚至几百个函数定义，它们是整个 C++ 程序的主体。
- **库函数：**原则上说系统提供的库函数与用户定义的函数并无区别，因此，也可以把用户定义函数和库函数归为一类。
- **全局说明：**由各个函数共同使用的常量、变量、用户定义的类型（也包括类）的说明以及已定义的函数的原型组成。
- **预处理命令：**
- **注释：**



C++程序的基本框架： SP/OOP框架

- SP 方法的缺点：
 - 没有充分利用 C++语言所提供的有力手段。类和对象是 C++语言的主要特征，但在 SP 框架下，类只起到一个配角的作用，它可能出现在全局说明中，作为一种新的用户定义的数据类型出现，而类作为程序模块划分的作用却未得到利用。
 - 以函数为中心对程序进行**模块划分**，主要是依照程序模块的功能特征，其划分具有相当大的**随意性**。在大多数情况下，如果划分不科学、不合理，自然会影响到整个程序的可读性、可维护性、可重用性等。
 - C++语言中的函数**没有层次关系**，除了主函数之外，所有的函数都是“平等”的，可以说是一个无序的集合。由这样一些函数组成的模块，其本身没有必然的强内聚性。函数间、模块间的联系较多，不利于程序的编制、调试、修改、扩充和重用。

C++程序的基本框架： SP/OOP框架

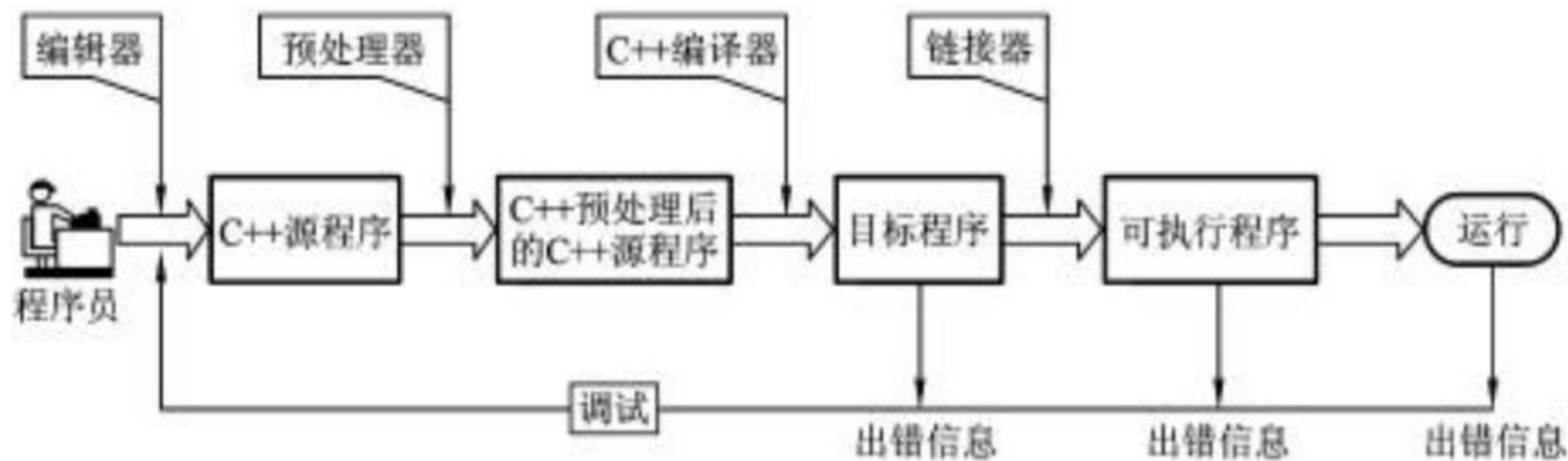
- 面向对象程序设计(OOP)思想设计的程序，它的主体是若干类定义的集合。
- 面向对象的程序设计把数据和对数据进行处理的过程作为一个整体——对象（Object）。
- 每一个对象是类的一个实例，类是同一类对象的抽象，可以把类比作图纸，对象就是按图纸和具体要求生产的一个个零件。
- 对象之间通过信息传递相互作用，形成有机的整体，所传递的信息称为消息（message）
- 类、对象、消息是面向对象程序设计的 3 个基本概念。

目录

- Hello World
 - 安装运行环境：vscode/docker/linux+g++/win+wsl2
 - 基本元素：注释/头文件/函数/语句/标准输入输出流
 - 基本符号与ASCII码
 - C++的词汇列表：关键字/标志符/字面常量/运算符/分隔符/
 - C++程序的基本框架：主函数/预处理命令/SP/OOP框架
 - 运行C++程序：编辑-预处理-编译-链接-调试
- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

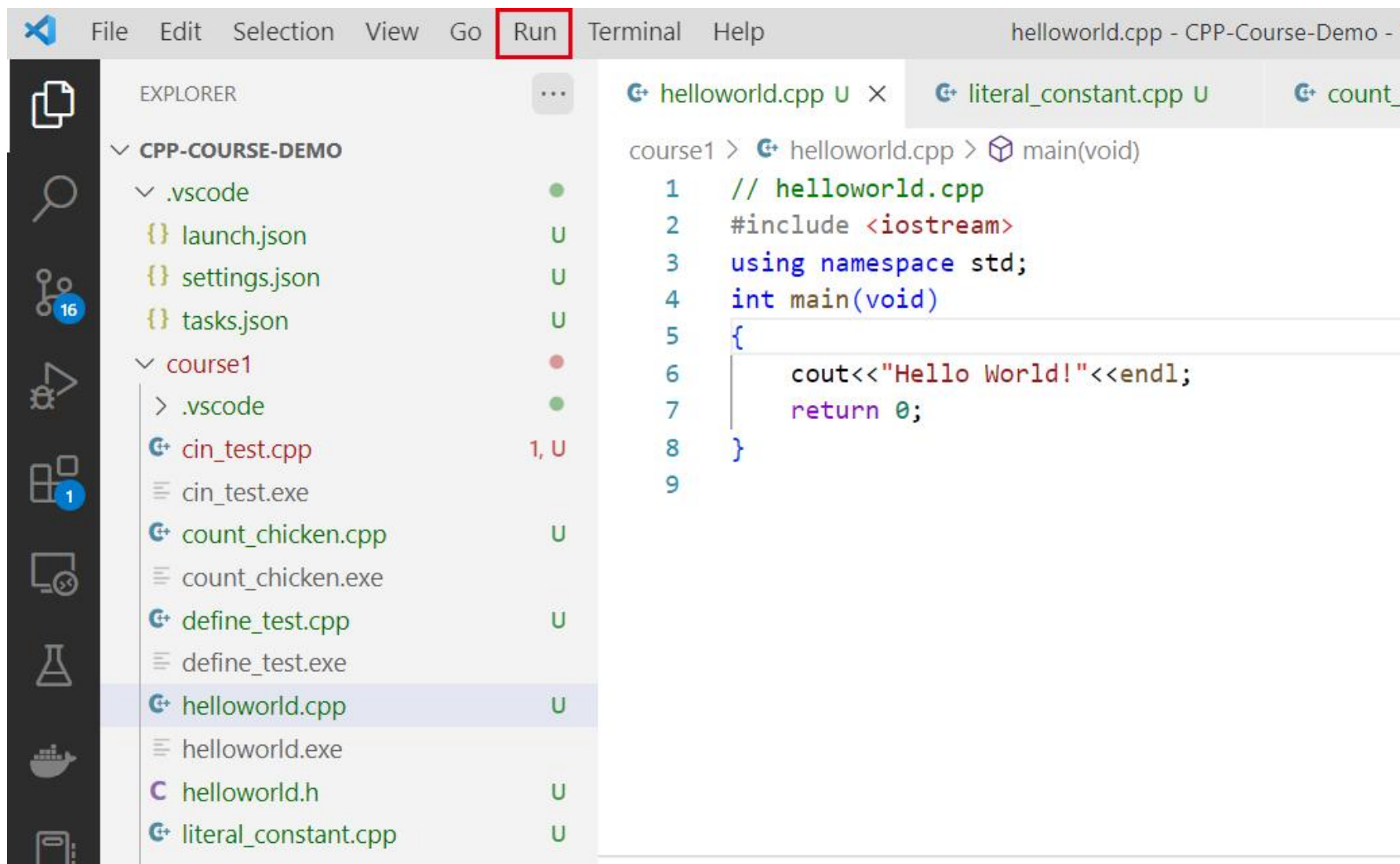
运行C++程序

- 从 C++源程序到可执行的机器语言程序还有三个步骤必须完成：编译预处理、编译、链接。
- 完成C++程序完整的步骤应包括：编辑、预处理、编译、链接和调试。
- “C++源程序”为.cpp 文件，“目标程序”为.obj文件，“可执行程序”为.exe 文件。



运行C++程序

- 如何通过VSCode来运行和调试C++程序



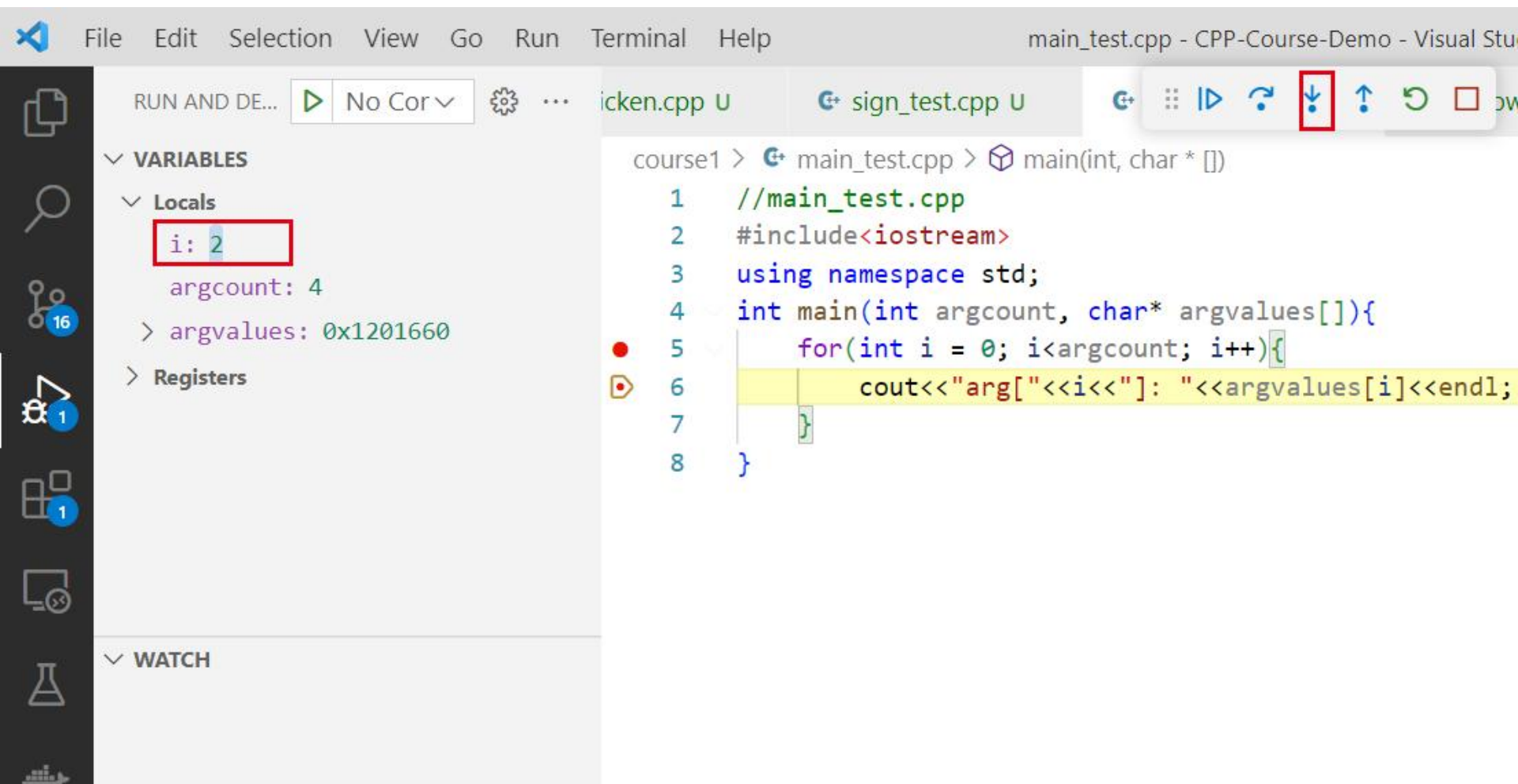
运行C++程序

- 如何通过VSCode来运行和调试C++程序。



运行C++程序

- 如何通过VSCode来运行和调试C++程序



思考题

- 1. C++的基本符号有哪些，分为哪几类？
- 2. 什么是关键字，什么是标识符，怎样构成合法的标识符？
- 3. 什么是常量，C++中常量的表示形式有哪些，C++的字面常量有哪几类？
- 4. 字符常量在机器内部如何存储，它与整数有什么关系？
- 5. 字符常量与字符串常量有什么区别，'a'与"a"有什么区别？
- 6. 以反斜杠开头的“字符”（也称为转义字符）主要用来表示什么样的字符，是否任何一个字符都可以
- 用这种方式来表示，如何表示？
- 7. 从结构程序设计的观点看，C++程序是由哪几部分组成的，它们之间的关系如何？
- 8. C++程序的主函数与其他函数有什么不同，最常用的 main 函数的使用格式是什么样的？

思考题

- 9. 什么是预处理，预处理在 C++ 程序中起什么作用？
- 10. 文件嵌入命令的作用是什么，什么是宏定义命令？
- 11. 程序中使用的 `cout` 代表什么，“<<”是用来完成什么功能的运算符？
- 12. 程序中使用的 `cin` 代表什么，“>>”是用来完成什么功能的运算符？
- 13. 什么情况下，程序中必须使用“`#include<iostream>`”预处理命令？
- 14. C++ 注释的格式有哪几种形式，如何使用？
- 15. 简述结构程序设计（SP）的优缺点。
- 16. OOP 框架的 C++ 程序是由哪几部分组成的，与 SP 比较 OOP 有哪些优越之处？
- 17. 简述从源程序到程序运行中间的各个处理步骤。

思考题

- (1) 标识符中，() 符号不能组成标识符。
 - A. 分割符 B. 下划线 C. 大小写字母 D. 数字字符
- (2) 下面字符串中，() 是不合法的标识符。
 - A. Pad B. max C. P#d D. t1mp
- (3) 下面字符串中，() 是合法的标识符。
 - A. b-b B. CCP C. scanf D. *jer
- (4) 下面的() 不是转义字符。
 - A. \\ B. \t C. \x11 D. \ff
- (5) 预处理命令都是以() 开头的。
 - A. * B. # C. : D. /